Kisaan Mitra AI
Application Architecture

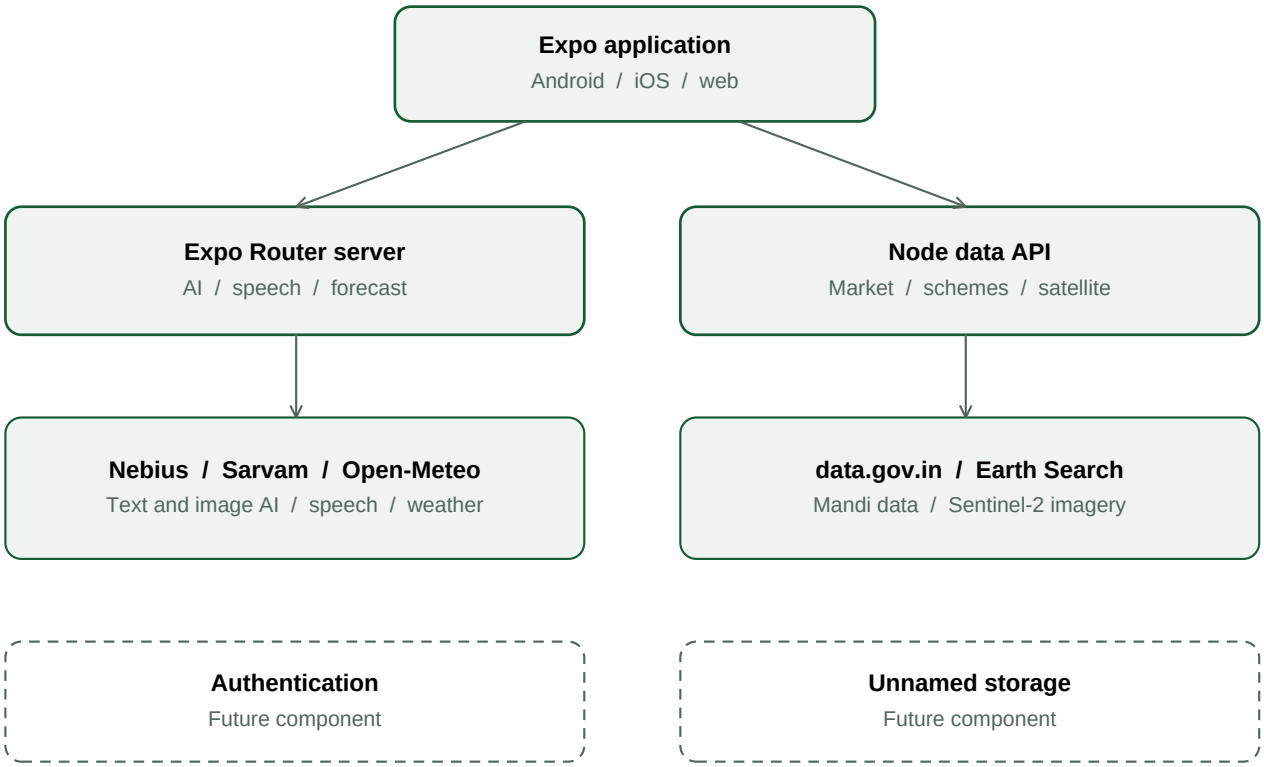
System structure and future integration boundaries

Version 1.0 | 6 November 2026

Kisaan Mitra AI is a mobile-first farming companion for Android, iOS, and web. Its current architecture combines an Expo client, Expo Router server routes, and a separate Node data API. This document explains the components, request flows, deployment boundaries, and current implementation limits.

Authentication and **Unnamed storage** are future components. No provider, storage engine, authentication method, or hosting platform is selected for them.

System context



Solid connections show current service paths. Dashed boxes identify future components. Both server components will need authentication and ownership checks before accessing user-owned data; future integration contracts remain undecided.

Additional current paths: the weather screen calls Open-Meteo directly; optional Google Maps integrations display and select farm locations; the Node API reads the reviewed schemes catalogue from backend/schemes.json.

Application components

The client uses Expo SDK 58, React Native 0.88, React 19.3, Expo Router, and TypeScript. The data API uses Node.js HTTP handling, geotiff, and proj4. Versions describe the current dependency manifest.

Component	Responsibility	Source
Routing and screens	Onboarding, dashboard, feature composition, responsive navigation	src/app/_layout.tsx; src/app/[screen].tsx
Shared interface and languages	Reusable controls and translations for six app languages	src/ui.tsx; src/i18n.tsx
Farm profile	Shared farm details, updates, and web profile persistence	src/farm-profile.tsx
Voice access and commands	Recording, dictation, narration, action matching, scoped confirmation	src/speech-access.tsx; src/voice-command-engine.ts
Guided registration	Questions, answer validation, corrections, map pin, review and confirmation	src/registration-guide.ts; src/components/VoiceRegistrationGuide.tsx
Assistant and profit	Conversation UI, buyer offers, calculated comparisons, AI explanations	src/assistant.tsx; src/profit.ts
Crop analysis	Photo selection, resizing, diagnosis request, and result display	src/crop-scan.tsx; src/diagnosis.ts
Client data access	Feature requests, loading, errors, and refresh behavior	src/services/; src/hooks/
Server integrations	AI and speech routes; separate market and satellite processing	src/app/api/; backend/server.mjs

Client composition

The root layout composes language, farm-profile, safe-area, speech, and voice-command providers around the Router stack. Most feature state remains in React context or local component state. The language preference is session state; the farm profile is additionally persisted in browser local storage on web.

Navigation boundary

The index route redirects to /splash. The dynamic [screen] route selects content such as /dashboard, /weather, /market, /map, /assistant, and /profile. Shared components and utilities stay outside src/app. Compact screens use bottom navigation; wider layouts use a side menu.

Voice and assistant flows

Spoken farming question

1. Capture. The client obtains microphone permission and records audio. It submits multipart audio and the selected language to POST /api/transcribe.

2. Transcribe. The Router server calls Sarvam and returns a transcript and language. The selected language takes precedence over an incorrect provider language detection.

3. Prepare context. The client sends the conversation and farm context to POST /api/chat. The server keeps the last ten valid messages and fetches Open-Meteo weather when the saved coordinates are valid.

4. Generate a response. The server assesses weather freshness and calls Nebius with the requested language, farm details, and any calculated profit plan. It returns a reply and optional profit guidance.

5. Speak when requested. The client calls POST /api/speak. Sarvam generates audio, which the client plays. Longer narration is split into provider-sized text chunks.

Field dictation

Dictation targets a specific field for the lifetime of a recording. Numeric fields normalize Indic digits and supported Telugu or English digit words, including spoken decimal points. Existing field limits and validation still apply. Names and places retain their free-text meaning.

Commands and confirmation

Commands use a deterministic action registry rather than an unrestricted AI agent. Known route phrases, visible button labels, aliases, numbered choices, and defined ordinals map to registered actions. Commands can also be typed without a microphone.

Only actions in the active screen or deepest open dialog are available. Ambiguous matches are rejected. A submission requires a separate confirmation, which expires after two minutes. Relevant edits, scope changes, action replacement or removal, and another command revoke a pending confirmation.

Numbered choices accept supported digit characters and defined ordinals. Spoken Telugu number words for numbered choices remain a vocabulary gap; numeric-field dictation support does not automatically extend command matching.

Guided onboarding

Registration collects name, state, district, village, crop, and area, then requires an exact map pin and explicit review confirmation. Location changes clear dependent place details and coordinates. The current mobile-code sign-in is a demonstration: no SMS is sent and no trusted server identity is established.

Farm intelligence flows

Crop photo assessment

The farmer captures or selects a crop photo. The client creates a JPEG with a maximum side of 1024 pixels and compression of 0.7 to reduce upload cost. It sends the base64 image and farm context to POST /api/diagnose.

The Router server invokes the Nebius vision model and normalizes its response. The client displays crop and disease descriptions, photo quality, severity, symptoms, alternative possibilities, suggested actions, and expert-consultation guidance. There is no durable account-level scan history today. Model output is guidance, not a confirmed field diagnosis.

Satellite vegetation analysis

The client sends the confirmed farm pin and area to the Node API. The server estimates a square field, searches the previous 90 days of Sentinel-2 L2A imagery through Element84 Earth Search, and groups candidate scenes into ten-day periods.

The server reads red and near-infrared raster bands and uses scene-classification data when available to exclude invalid pixels. It calculates NDVI observations, the latest vegetation-vigor value, and change from the previous observation. Successful analysis results are cached in process memory for three hours.

One pin and total area cannot represent an irregular plot boundary. Cloud cover, missing imagery, and raster failures can prevent a result. NDVI describes vegetation condition and does not diagnose disease. Google Maps is an optional display and location-selection integration; it is not the satellite-analysis data source.

Profit planning

The farmer supplies expected saleable harvest, cultivation costs, recent buyer offers, and selling costs. The server validates quantities and quote dates, calculates net profit and break-even prices, ranks the compared options, and evaluates a ten-percent lower-price scenario.

Nebius explains the server-calculated results and supplies practical checklists. Model output cannot override authoritative ranks or monetary figures. Missing data prompts completion of the form; the server does not invent financial inputs. Offers are farmer-entered rather than verified live quotes, and a comparison is not a guarantee of future profit.

Weather market and schemes

The weather screen currently requests Open-Meteo for fixed Anakapalle coordinates, while assistant weather uses the saved farm pin. Mandi data comes through the Node API from data.gov.in; the server currently filters paddy or rice records. Government schemes come from a reviewed JSON catalogue, with state matching and crop relevance. Some client queries still use demo profile or crop defaults.

API boundaries

Two separate API processes are implemented. The Router server owns AI and speech integrations; the Node API owns market data, the schemes catalogue, and satellite computation. Authentication and Unnamed storage have no implemented API contracts yet.

Expo Router server routes

Method and route	Input	Result
GET /api/weather	No client query parameters	Forecast for configured coordinates through Open-Meteo
POST /api/transcribe	Multipart audio; optional language	Transcript and language through Sarvam
POST /api/speak	Text and language	Encoded speech audio through Sarvam
POST /api/chat	Messages, language, farm context; optional profit inputs	Reply; optional calculated profit plan and model guidance
POST /api/diagnose	Base64 image and farm context	Normalized crop assessment through Nebius

Node data API routes

Method and route	Input	Result
GET /api/health	None	Process health response
GET /api/market/prices	State; client also sends commodity	Paddy or rice mandi records through data.gov.in
GET /api/schemes	State and crop	Matching reviewed catalogue entries, ranked by relevance
POST /api/satellite/analysis	Latitude, longitude, farm area	NDVI trend and latest vegetation-vigor estimate

Failure handling and performance

Provider failures produce recoverable loading or error states, and selected screens use labeled sample content. Requests have provider-specific timeouts. The transcription endpoint limits uploads to five million bytes; field dictation records for up to thirty seconds. Client image preparation reduces mobile upload sizes.

Weather forecasts are cached for thirty minutes, market responses for fifteen minutes, and successful satellite analyses for three hours. Caches live in server process memory and are lost on restart; replicas do not share entries. Satellite raster work runs in the Node request process. Background jobs or bounded processing concurrency may be needed as usage grows, but no queue is implemented.

Authentication and Unnamed storage

Both components will be built later. The boundaries below preserve their place in the architecture without choosing a provider, database technology, schema, or authentication mechanism.

Authentication

Authentication will establish trusted identity and sessions. The demonstration verification flow will be replaced. Server routes will validate identity before authorizing access to user-owned information; a farmer name or mobile number in a request body is not sufficient proof of identity.

The login method, verification mechanism, session format, account recovery process, and integration contract remain open decisions. Both server components must apply consistent ownership rules when they receive user-specific operations.

Unnamed storage

Unnamed storage will provide durable persistence. Server-side application code should own reads and writes and enforce ownership using validated Authentication context. The client should receive only the information needed for its authorized operations.

Information	Current lifecycle	Future responsibility
Identity and sessions	Local demonstration sign-in state	Authentication
Farmer profile and farm pin	React context; browser local storage on web	Unnamed storage
Language preference	Client session context	Unnamed storage if account sync is required
Conversation and profit plans	Client session state and request payloads	Unnamed storage if history is enabled
Crop images and assessments	Temporary client data and provider processing	Unnamed storage only if retention is introduced
Tasks and checklist progress	Local interaction state	Unnamed storage
Community activity	Demonstration likes, replies, connection requests	Unnamed storage if a real community service is built

Decisions before persistence

Define record ownership, retention, deletion, export, backups, and cross-device synchronization before introducing durable histories. Farm records, task progress, and optional scan or conversation histories are candidate responsibilities, not an approved retention policy. A shared identity and ownership contract should precede feature-specific persistence.

Deployment and implementation priorities

app.json sets Expo web output to server. Router API routes therefore require a server-capable deployment. The Node data API runs separately. Native clients need reachable origins; localhost on a physical phone refers to that phone.

Configuration	Purpose and boundary
EXPO_PUBLIC_APP_API_BASE_URL	Public app-server origin used explicitly by native speech and chat requests
EXPO_PUBLIC_API_BASE_URL	Public Node API origin; default http://localhost:4000
NEBIUS_API_KEY / SARVAM_API_KEY	Router server secrets for AI and speech
DATA_GOV_IN_API_KEY	Node server secret for mandi requests
Google Maps keys / PORT	Optional platform-restricted map keys; Node port defaults to 4000

The root environment configures Expo and its server routes; the data API run command reads backend/.env. Server credentials must never enter public client variables. Use HTTPS in deployment and for browser microphone access. EAS cloud builds and updates can be configured later; no eas.json profile is currently defined.

Production work

Identity and access. Add Authentication and ownership checks before user-specific persistence. Review request limits, provider-cost controls, and operational monitoring. The Node API currently allows any CORS origin and only the content-type request header; future authentication headers require a corresponding CORS review.

Profile consistency. Align weather, market, and scheme queries with saved farm details. Verify server addressing on deployed Android and iOS: speech and chat support an explicit app-server origin, while diagnosis still requests a relative route.

Data and scaling. Define Unnamed storage lifecycle policies, then add durable workflows. Community connections and notification views currently do not provide real expert messaging or production delivery. Review satellite processing capacity as usage increases.

Verification and implementation references

Regression tests cover voice services, command matching and confirmation, guided registration, and profit calculations using isolated provider mocks. They do not establish live microphone accuracy, real-provider availability, or complete native deployment behavior.

Checks: node --test tests/*.test.mjs; npx expo lint; npx tsc --noEmit.

Source references: README.md; package.json; app.json; src/app/_layout.tsx; src/app/[screen].tsx; src/app/api/; backend/server.mjs; src/farm-profile.tsx; src/services/; src/voice-command-engine.ts; src/profit.ts.

Repository: github.com/navya504/kissan-mitra-ai